

sync-lock-impl

too much milk

roommates Alice and Bob want to keep fridge stocked with milk:

time	Alice	Bob
3:00	look in fridge. no milk	
3:05	leave for store	
3:10	arrive at store	look in fridge. no milk
3:15	buy milk	leave for store
3:20	return home, put milk in fridge	arrive at store
3:25		buy milk
3:30		return home, put milk in fridge

how can Alice and Bob coordinate better?

too much milk “solution” 1 (algorithm)

leave a note: “I am buying milk”

place before buying, remove after buying

don't try buying if there's a note

≈ setting/checking a variable (e.g. “note = 1”)
with atomic load/store of variable

```
if (no milk) {  
    if (no note) {  
        leave note;  
        buy milk;  
        remove note;  
    }  
}
```

exercise: why doesn't this work?

too much milk “solution” 1 (timeline)

Alice

```
if (no milk) {  
    if (no note) {  
  
        leave note;  
        buy milk;  
        remove note;  
  
    }  
}
```

Bob

```
if (no milk) {  
    if (no note) {  
  
        leave note;  
        buy milk;  
        remove note;  
  
    }  
}
```

too much milk “solution” 2 (algorithm)

intuition: leave note when buying *or checking if need to buy*

```
leave note;  
if (no milk) {  
    if (no note) {  
        buy milk;  
    }  
}  
remove note;
```


too much milk: “solution” 2 (timeline)

too much milk: “solution” 2 (timeline)

Alice

```
leave note;
```

```
if (no milk) {
```

```
    if (no note) {  
        buy milk;
```

```
    }
```

```
}
```

```
remove note;
```

too much milk: “solution” 2 (timeline)

Alice

```
leave note;
```

```
if (no milk) {
```

```
    if (no note) {
```

```
        buy milk;
```

```
    }
```

```
}
```

```
remove note;
```



but there's *always a note*

too much milk: “solution” 2 (timeline)

Alice

```
leave note;
```

```
if (no milk) {
```

```
    if (no note) {
```

```
        buy milk,
```

```
    }
```

```
}
```

```
remove note;
```



but there's *always a note*

...will never buy milk (twice *or* once)

“solution” 3: algorithm

intuition: label notes so Alice knows which is hers (and vice-versa)

computer equivalent: separate noteFromAlice and noteFromBob variables

Alice

```
leave note from Alice;  
if (no milk) {  
    if (no note from Bob) {  
        buy milk  
    }  
}  
remove note from Alice;
```

Bob

```
leave note from Bob;  
if (no milk) {  
    if (no note from Alice) {  
        buy milk  
    }  
}  
remove note from Bob;
```

too much milk: "solution" 3 (timeline)

Alice

leave note from Alice

if (no milk) {

 if (no note from Bob) {

 buy ~~m~~ilk

 }

}

remove note from Alice

Bob

leave note from Bob

if (no milk) {

 if (no ~~n~~ote from Alice) {

 buy ~~m~~ilk

 }

}

remove note from Bob

too much milk: is it possible

is there a solutions with writing/reading notes?

≈ loading/storing from shared memory

yes, but it's not very elegant

too much milk: solution 4 (algorithm)

Alice

```
leave note from Alice
while (note from Bob) {
    do nothing
}
if (no milk) {
    buy milk
}
remove note from Alice
```

Bob

```
leave note from Bob
if (no note from Alice) {
    if (no milk) {
        buy milk
    }
}
remove note from Bob
```

exercise (hard): prove (in)correctness

exercise (hard): extend to three people

Peterson's algorithm

general version of solution

see, e.g., Wikipedia

we'll use special hardware support instead

x86-64 spinlock with xchg

lock variable in shared memory: the_lock

if 1: someone has the lock; if 0: lock is free to take

acquire:

```
    movl $1, %eax           // %eax <- 1
    lock xchg %eax, the_lock // swap %eax and the_lock
                           // sets the_lock to 1 (taken)
                           // sets %eax to prior val. of the_lock

    test %eax, %eax         // if the_lock wasn't 0 before:
    jne acquire             //   try again
    ret
```

release:

```
    mfence                 // for memory order reasons
    movl $0, the_lock      // then, set the_lock to 0 (not taken)
    ret
```

x86-64 spinlock with xchg

lock variable in shared memory: the_lock

if 1: someone has the lock; if 0: lock is free to take

acquire:

```
    movl $1, %eax           // %eax <- 1
    lock xchg %eax, the_lock // swap %eax and the_lock
                           // sets the_lock to 1 (taken)
                           // sets %eax to prior val. of the_lock

    test %eax, %eax         // if the_lock wasn't 0 before:
    jne acquire             //   try again
    ret
```

release:

```
    mfence                 // for memory order reasons
    movl $0, the_lock      // then, set the_lock to 0 (not taken)
    ret
```


x86-64 spinlock with xchg

lock variable in shared memory: `the_lock`

if 1: someone has the lock; if 0: lock is free to take

acquire:

```
movl $1, %eax           // %eax <- 1
lock xchg %eax, the_lock // swap %eax and the_lock
                        // sets the_lock to 1 (taken)
                        // sets %eax to prior val. of the_lock
test %eax, %eax          // if the_lock wasn't 0 before:
jne acquire              // try again
ret
```

set lock variable to 1 (taken)
read old value

release:

```
mfence                  // for memory order reasons
movl $0, the_lock       // then, set the_lock to 0 (not taken)
ret
```

x86-64 spinlock with xchg

lock variable in shared memory: the_lock

if 1: someone has the lock; if 0: lock is free to take

acquire:

```
    movl $1, %eax           // %eax <- 1
    lock xchg %eax, the_lock // swap %eax and the_lock
                             // sets the_lock to 1 (taken)
                             // sets %eax to prior val. of the_lock
    test %eax, %eax         // if the_lock wasn't 0 before:
    jne acquire             // try again
    ret
```

if lock was already locked retry
“spin” until lock is released elsewhere

release:

```
    mfence                 // for memory order reasons
    movl $0, the_lock      // then, set the_lock to 0 (not taken)
    ret
```

x86-64 spinlock with xchg

lock variable in shared memory: the_lock

if 1: someone has the lock; if 0: lock is free to take

acquire:

```
    movl $1, %eax           // %eax <- 1
    lock xchg %eax, the_lock // swap %eax and the_lock
                             // sets the_lock to 1 (taken)
                             // sets %eax to prior val. of the_lock
    test %eax, %eax         // if the_lock wasn't 0 before:
    jne acquire             // try again
    ret
```

release lock by setting it to 0 (not taken)
allows looping acquire to finish

release:

```
    mfence                 // for memory order reasons
    movl $0, the_lock      // then, set the_lock to 0 (not taken)
    ret
```

x86-64 spinlock with xchg

lock variable in shared memory: the_lock

if 1: someone has the lock; if 0: lock is free to take

acquire:

```
    movl $1, %eax           // %eax <- 1
    lock xchg %eax, the_lock // swap %eax and the_lock
                             // sets the_lock to 1 (taken)
                             // sets %eax to prior val. of the_lock
    test %eax, %eax         // if the_lock wasn't 0 before:
    jne acquire             // try again
    ret
```

release:

```
    mfence                 // for memory order reasons
    movl $0, the_lock      // then, set the_lock to 0 (not taken)
    ret
```

Intel's manual says:
no reordering of loads/stores across a lock
or mfence instruction

exercise: spin wait

consider implementing 'waiting' functionality of pthread_join

thread calls ThreadFinish() when done

complete code below:

```
finished: .quad 0
```

```
ThreadFinish:
```

```
-----  
ret
```

```
ThreadWaitForFinish:
```

```
-----  
lock xchg %eax, finished  
cmp $0, %eax  
---- ThreadWaitForFinish  
ret
```

A. mfence; mov \$1, finished	C. mov \$0, %eax	E. je
B. mov \$1, finished; mfence	D. mov \$1, %eax	F. jne

exercise: spin wait

```
finished: .quad 0
```

```
ThreadFinish:
```

```
-----A-----  
ret
```

```
ThreadWaitForFinish:
```

```
-----B-----  
lock xchg %eax, finished  
cmp $0, %eax  
__C_ ThreadWaitForFinish  
ret
```

```
/* or without using a writing instruction: */
```

```
mov %eax, finished
```

```
mfence
```

```
cmp $0, %eax
```

```
je ThreadWaitForFinish
```

```
ret
```

A. mfence; mov \$1, finished	C. mov \$0, %eax	E. je
B. mov \$1, finished; mfence	D. mov \$1, %eax	F. jne

againframe(exerSpinWaitSoln)

spinlock problems

lock abstraction is not powerful enough

- lock/unlock operations don't handle "wait for event"

- common thing we want to do with threads

- solution: other synchronization abstractions

spinlocks waste CPU time more than needed

- want to run another thread instead of infinite loop

- solution: lock implementation integrated with scheduler

spinlocks can send a lot of messages on the shared bus

- more efficient atomic operations to implement locks

spinlock problems

lock abstraction is not powerful enough

- lock/unlock operations don't handle "wait for event"

- common thing we want to do with threads

- solution: other synchronization abstractions

spinlocks waste CPU time more than needed

- want to run another thread instead of infinite loop

- solution: lock implementation integrated with scheduler

spinlocks can send a lot of messages on the shared bus

- more efficient atomic operations to implement locks

spinlock problems

lock abstraction is not powerful enough

- lock/unlock operations don't handle "wait for event"

- common thing we want to do with threads

- solution: other synchronization abstractions

spinlocks waste CPU time more than needed

- want to run another thread instead of infinite loop

- solution: lock implementation integrated with scheduler

spinlocks can send a lot of messages on the shared bus

- more efficient atomic operations to implement locks

spinlock problems

lock abstraction is not powerful enough

- lock/unlock operations don't handle "wait for event"

- common thing we want to do with threads

- solution: other synchronization abstractions

spinlocks waste CPU time more than needed

- want to run another thread instead of infinite loop

- solution: lock implementation integrated with scheduler

spinlocks can send a lot of messages on the shared bus

- more efficient atomic operations to implement locks

problem: busy waits

```
while(xchg(&lk->locked, 1) != 0)  
    ;
```

what if it's going to be a while?

waiting for process that's waiting for I/O?

really would like to do something else with CPU instead...

problem: busy waits

```
while(xchg(&lk->locked, 1) != 0)  
    ;
```

what if it's going to be a while?

waiting for process that's waiting for I/O?

really would like to do something else with CPU instead...

mutexes: intelligent waiting

want: locks that wait better

example: POSIX mutexes

instead of running infinite loop, give away CPU

lock = go to sleep, add self to list

sleep = scheduler runs something else

unlock = wake up sleeping thread

mutexes: intelligent waiting

want: locks that wait better

example: POSIX mutexes

instead of running infinite loop, give away CPU

lock = go to sleep, add self to list

sleep = scheduler runs something else

unlock = wake up sleeping thread

better lock implementation idea

shared list of waiters

spinlock protects list of waiters from concurrent modification

lock = use spinlock to add self to list, then wait without spinlock

unlock = use spinlock to remove item from list

better lock implementation idea

shared list of waiters

spinlock protects list of waiters from concurrent modification

lock = use spinlock to add self to list, then wait without spinlock

unlock = use spinlock to remove item from list

one possible implementation

```
struct Mutex {
    SpinLock guard_spinlock;
    bool lock_taken = false;
    WaitQueue wait_queue;
};

LockMutex(Mutex *m) {
    LockSpinlock(&m->guard_spinlock);
    if (m->lock_taken) {
        put current thread on m->wait_queue
        mark current thread as waiting
        /* xv6: myproc()->state = SLEEPING; */
        UnlockSpinlock(&m->guard_spinlock);
        run scheduler (context switch)
    } else {
        m->lock_taken = true;
        UnlockSpinlock(&m->guard_spinlock);
    }
}
```

```
UnlockMutex(Mutex *m) {
    LockSpinlock(&m->guard_spinlock);
    if (m->wait_queue not empty) {
        remove a thread from m->wait_queue
        mark thread as no longer waiting
        /* xv6: myproc()->state = RUNNABLE; */
    } else {
        m->lock_taken = false;
    }
    UnlockSpinlock(&m->guard_spinlock);
}
```

one possible implementation

```
struct Mutex {
    SpinLock guard_spinlock;
    bool lock_taken = false;
    WaitQueue wait_queue;
};

LockMutex(Mutex *m) {
    LockSpinlock(&m->guard_spinlock);
    if (m->lock_taken) {
        put current thread on m->wait_queue
        mark current thread as waiting
        /* xv6: myproc()->state = SLEEPING; */
        UnlockSpinlock(&m->guard_spinlock);
        run scheduler (context switch)
    } else {
        m->lock_taken = true;
        UnlockSpinlock(&m->guard_spinlock);
    }
}

UnlockMutex(Mutex *m) {
    LockSpinlock(&m->guard_spinlock);
    if (m->wait_queue not empty) {
        remove a thread from m->wait_queue
        mark thread as no longer waiting
        /* xv6: myproc()->state = RUNNABLE; */
    } else {
        m->lock_taken = false;
    }
    UnlockSpinlock(&m->guard_spinlock);
}
```

spinlock protecting lock_taken and wait_queue
only held for very short amount of time (compared to mutex itself)

one possible implementation

```
struct Mutex {
    SpinLock guard_spinlock;
    bool lock_taken = false;
    WaitQueue wait_queue;
};

LockMutex(Mutex *m) {
    LockSpinlock(&m->guard_spinlock);
    if (m->lock_taken) {
        put current thread on m->wait_queue
        mark current thread as waiting
        /* xv6: myproc()->state = SLEEPING; */
        UnlockSpinlock(&m->guard_spinlock);
        run scheduler (context switch)
    } else {
        m->lock_taken = true;
        UnlockSpinlock(&m->guard_spinlock);
    }
}
```

```
UnlockMutex(Mutex *m) {
    LockSpinlock(&m->guard_spinlock);
    if (m->wait_queue not empty) {
        remove a thread from m->wait_queue
        mark thread as no longer waiting
        /* xv6: myproc()->state = RUNNABLE; */
    } else {
        m->lock_taken = false;
    }
    UnlockSpinlock(&m->guard_spinlock);
}
```

tracks whether any thread has locked and not unlocked

one possible implementation

```
struct Mutex {
    SpinLock guard_spinlock;
    bool lock_taken = false;
    WaitQueue wait_queue;
};

LockMutex(Mutex *m) {
    LockSpinlock(&m->guard_spinlock);
    if (m->lock_taken) {
        put current thread on m->wait_queue
        mark current thread as waiting
        /* xv6: myproc()->state = SLEEPING; */
        UnlockSpinlock(&m->guard_spinlock);
        run scheduler (context switch)
    } else {
        m->lock_taken = true;
        UnlockSpinlock(&m->guard_spinlock);
    }
}

UnlockMutex(Mutex *m) {
    LockSpinlock(&m->guard_spinlock);
    if (m->wait_queue not empty) {
        remove a thread from m->wait_queue
        mark thread as no longer waiting
        /* xv6: myproc()->state = RUNNABLE; */
    } else {
        m->lock_taken = false;
    }
    UnlockSpinlock(&m->guard_spinlock);
}
```

list of threads that discovered lock is taken
and are waiting for it be free
these threads are *not runnable*

one possible implementation

```
struct Mutex {
    SpinLock guard_spinlock;
    bool lock_taken = false;
    WaitQueue wait_queue;
};

LockMutex(Mutex *m) {
    LockSpinlock(&m->guard_spinlock);
    if (m->lock_taken) {
        put current thread on m->wait_queue
        mark current thread as waiting
        /* xv6: myproc()->state = SLEEPING; */
        UnlockSpinlock(&m->guard_spinlock);
        run scheduler (context switch)
    } else {
        m->lock_taken = true;
        UnlockSpinlock(&m->guard_spinlock);
    }
}
```

```
UnlockMutex(Mutex *m) {
    LockSpinlock(&m->guard_spinlock);
    if (m->wait_queue not empty) {
        remove a thread from m->wait_queue
        mark thread as no longer waiting
        /* xv6: myproc()->state = RUNNABLE; */
    } else {
        m->lock_taken = false;
    }
    UnlockSpinlock(&m->guard_spinlock);
}
```

instead of setting lock_taken to false
choose thread to hand-off lock to

one possible implementation

```
struct Mutex {
    SpinLock guard_spinlock;
    bool lock_taken = false;
    WaitQueue wait_queue;
};

LockMutex(Mutex *m) {
    LockSpinlock(&m->guard_spinlock);
    if (m->lock_taken) {
        put current thread on m->wait_queue
        mark current thread as waiting
        /* xv6: myproc()->state = SLEEPING; */
        UnlockSpinlock(&m->guard_spinlock);
        run scheduler (context switch)
    } else {
        m->lock_taken = true;
        UnlockSpinlock(&m->guard_spinlock);
    }
}
```

```
UnlockMutex(Mutex *m) {
    LockSpinlock(&m->guard_spinlock);
    if (m->wait_queue not empty) {
        remove a thread from m->wait_queue
        mark thread as no longer waiting
        /* xv6: myproc()->state = RUNNABLE; */
    } else {
        m->lock_taken = false;
    }
    UnlockSpinlock(&m->guard_spinlock);
}
```

subtly: if UnlockMutex runs here on another core
need to make sure scheduler on the other core doesn't switch to thread
while it is still running (would 'clone' thread/mess up registers)

one possible implementation

```
struct Mutex {
    SpinLock guard_spinlock;
    bool lock_taken = false;
    WaitQueue wait_queue;
};

LockMutex(Mutex *m) {
    LockSpinlock(&m->guard_spinlock);
    if (m->lock_taken) {
        put current thread on m->wait_queue
        mark current thread as waiting
        /* xv6: myproc()->state = SLEEPING; */
        UnlockSpinlock(&m->guard_spinlock);
        run scheduler (context switch)
    } else {
        m->lock_taken = true;
        UnlockSpinlock(&m->guard_spinlock);
    }
}
```

```
UnlockMutex(Mutex *m) {
    LockSpinlock(&m->guard_spinlock);
    if (m->wait_queue not empty) {
        remove a thread from m->wait_queue
        mark thread as no longer waiting
        /* xv6: myproc()->state = RUNNABLE; */
    } else {
        m->lock_taken = false;
    }
    UnlockSpinlock(&m->guard_spinlock);
}
```


mutex and scheduler subtlty

core 0 (thread A)	core 1 (thread B)
start LockMutex	
acquire spinlock	
discover lock taken	
enqueue thread A	
thread A set not runnable	
release spinlock	start UnlockMutex
	thread A set runnable
	finish UnlockMutex
	run scheduler
	scheduler switches to A
	... with old verison of registers
thread A runs scheduler	...
... finally saving registers	...

Linux soln.: track 'thread running' separately from 'thread runnable'

xv6 soln.: hold scheduler lock until thread A saves registers

mutex and scheduler subtlly

core 0 (thread A)	core 1 (thread B)
start LockMutex	
acquire spinlock	
discover lock taken	
enqueue thread A	
thread A set not runnable	
release spinlock	start UnlockMutex
	thread A set runnable
	finish UnlockMutex
	run scheduler
	scheduler switches to A
	<i>... with old verison of registers</i>
thread A runs scheduler	...
... finally saving registers	...

Linux soln.: track 'thread running' separately from 'thread runnable'

xv6 soln.: hold scheduler lock until thread A saves registers

mutex efficiency

‘normal’ mutex *uncontended* case:

lock: acquire + release spinlock, see lock is free

unlock: acquire + release spinlock, see queue is empty

not much slower than spinlock

implementing locks: single core

intuition: context switch only happens on interrupt
timer expiration, I/O, etc. causes OS to run

solution: disable them
reenable on unlock

x86 instructions:

`cli` — disable interrupts

`sti` — enable interrupts

naive interrupt enable/disable (1)

```
Lock() {  
    disable interrupts;  
}
```

```
Unlock() {  
    enable interrupts;  
}
```

problem: user can *hang the system*:

```
    Lock(some_lock);  
    while (true) {}
```

problem: can't do I/O within lock

```
    Lock(some_lock);  
    read from disk  
    /* waits forever for (disabled) interrupt  
       from disk IO finishing */
```


naive interrupt enable/disable (2)

```
Lock() {  
    disable interrupts;  
}
```

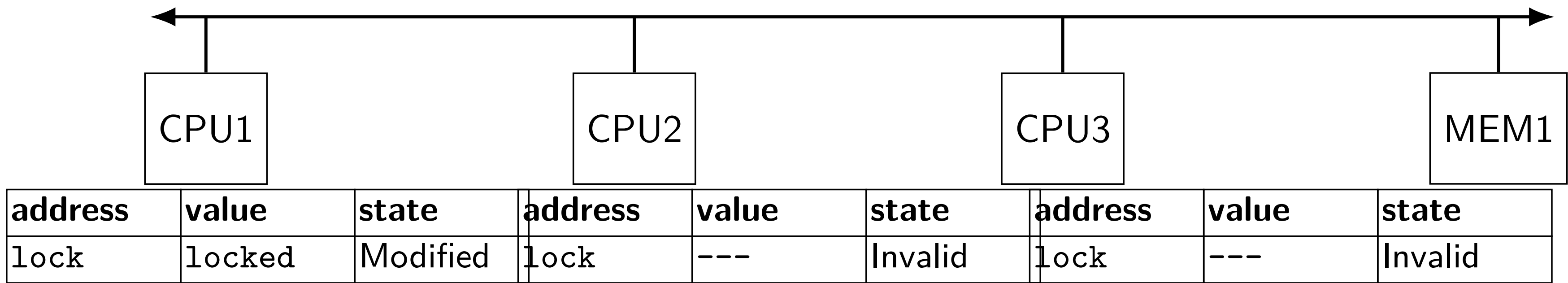
```
Unlock() {  
    enable interrupts;  
}
```

problem: nested locks

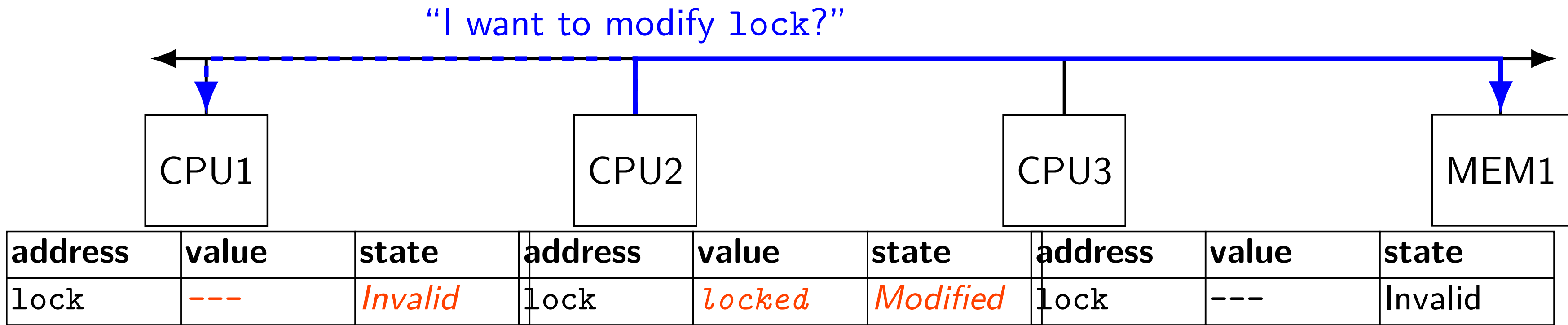
```
    Lock(milk_lock);  
if (no milk) {  
    Lock(store_lock);  
    buy milk  
    Unlock(store_lock);  
    /* interrupts enabled here?? */  
}  
Unlock(milk_lock);
```

ping-ponging

ping-ponging

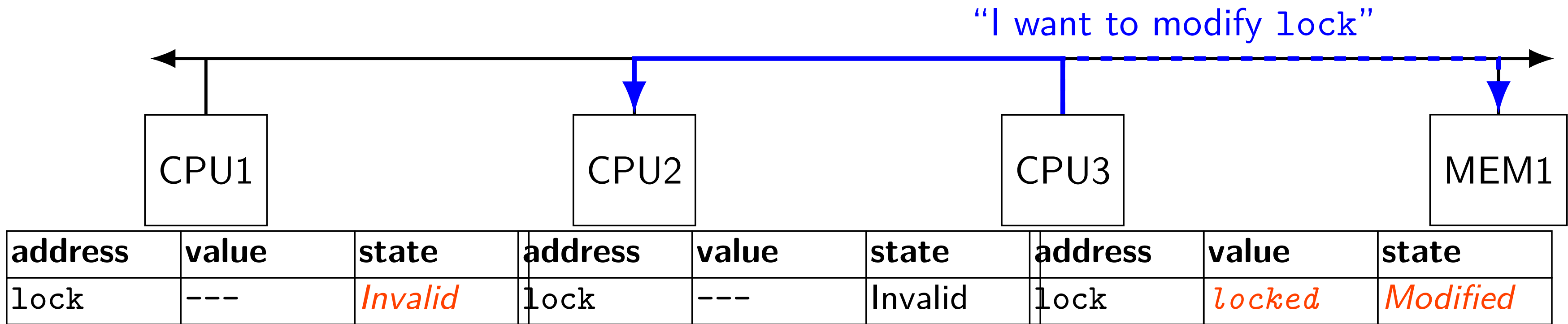


ping-ponging



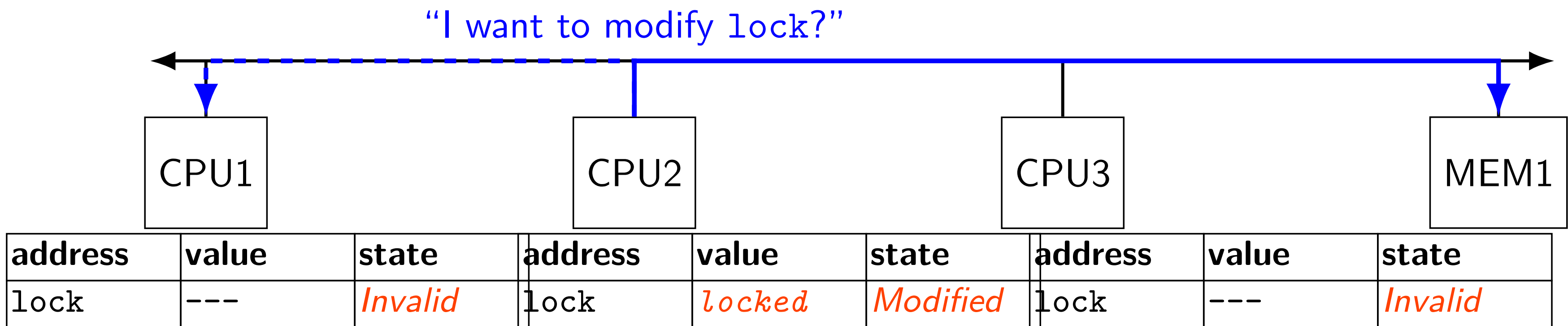
CPU2 read-modify-writes lock
(to see it is still locked)

ping-ponging



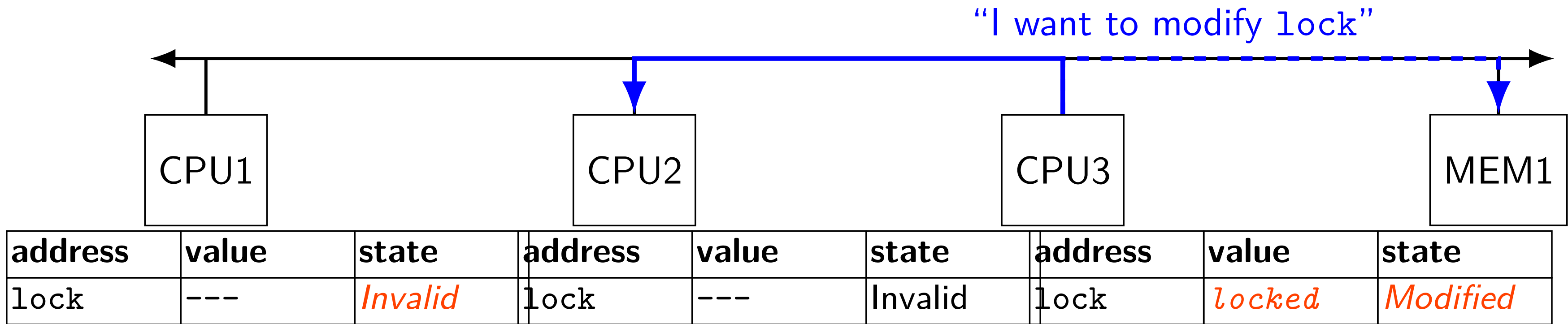
CPU3 read-modify-writes lock
(to see it is still locked)

ping-ponging



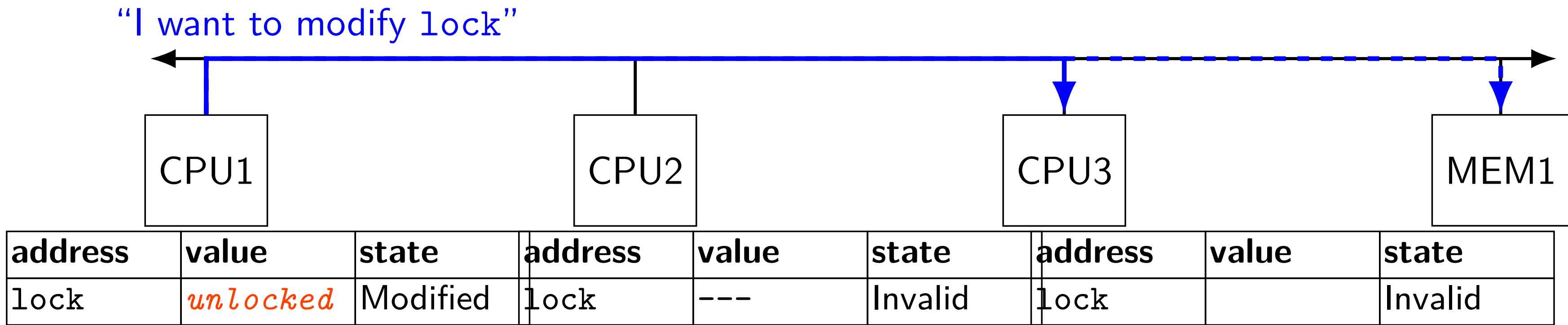
CPU2 read-modify-writes lock
(to see it is still locked)

ping-ponging



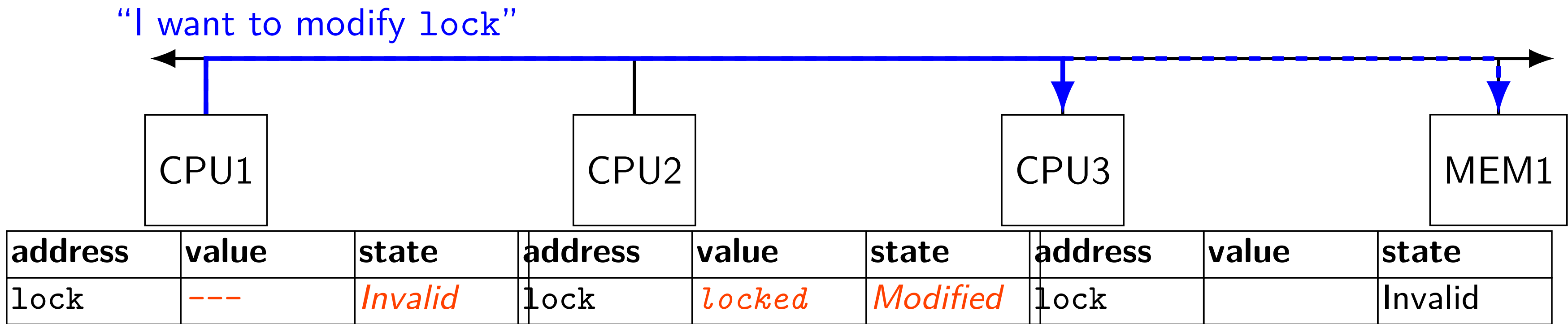
CPU3 read-modify-writes lock
(to see it is still locked)

ping-ponging



CPU1 sets lock to unlocked

ping-ponging



some CPU (this example: CPU2) acquires lock

ping-ponging

test-and-set problem: cache block “ping-pongs” between caches

- each waiting processor reserves block to modify

- could maybe wait until it determines modification needed — but not typical implementation

each transfer of block sends messages on bus

... so bus can't be used for real work

- like what the processor with the lock is doing

test-and-test-and-set (pseudo-C)

```
acquire(int *the_lock) {  
    do {  
        while (ATOMIC-READ(the_lock) == 0) { /* try again */ }  
    } while (ATOMIC-TEST-AND-SET(the_lock) == ALREADY_SET);  
}
```

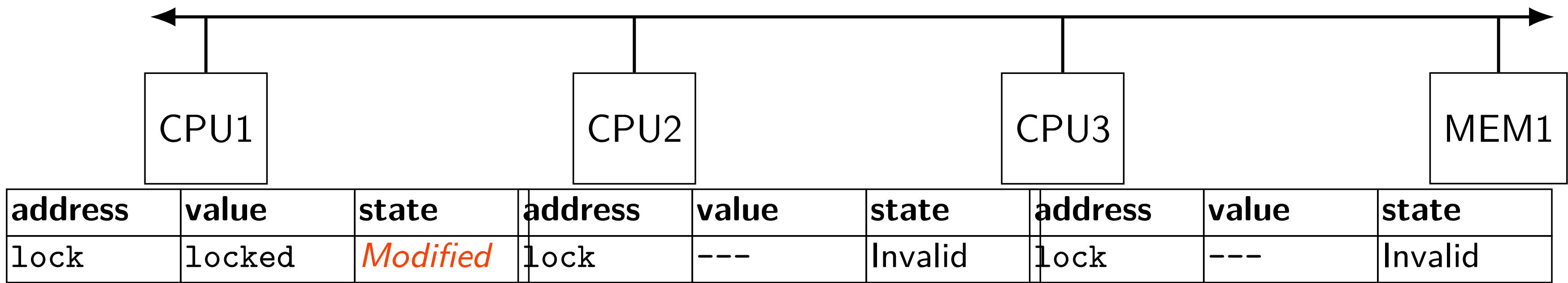

test-and-test-and-set (assembly)

acquire:

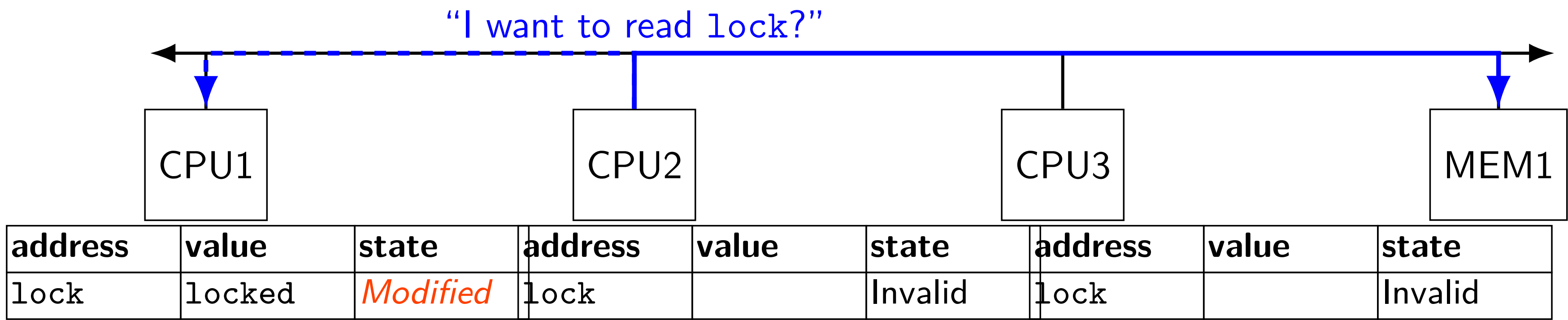
```
    cmp $0, the_lock          // test the lock non-atomically
                                // unlike lock xchg --- keeps lock in Shared state!
    jne acquire               // try again (still locked)
    // lock possibly free
    // but another processor might lock
    // before we get a chance to
    // ... so try with atomic swap:
    movl $1, %eax              // %eax <- 1
    lock xchg %eax, the_lock   // swap %eax and the_lock
                                // sets the_lock to 1
                                // sets %eax to prior value of the_lock
    test %eax, %eax            // if the_lock wasn't 0 (someone else got it first):
    jne acquire                //   try again
    ret
```

less ping-ponging

less ping-ponging

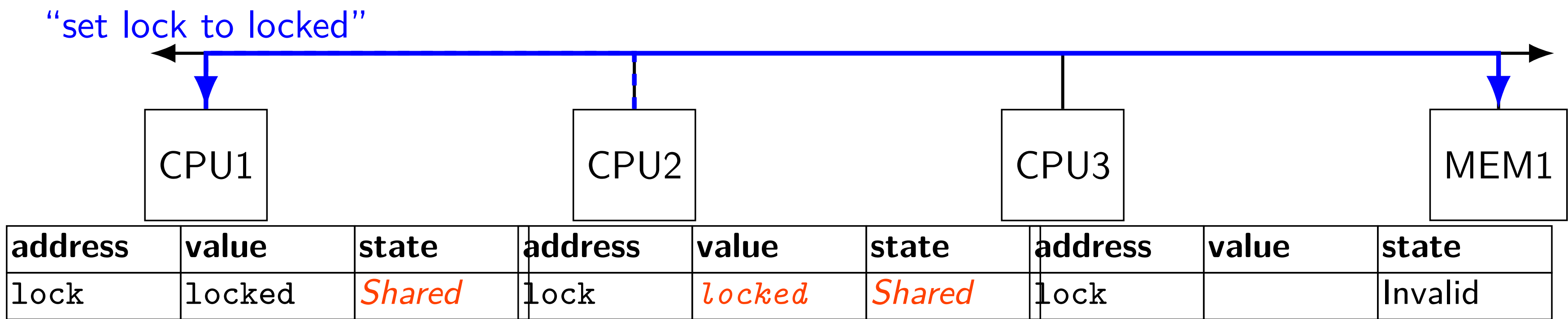


less ping-ponging



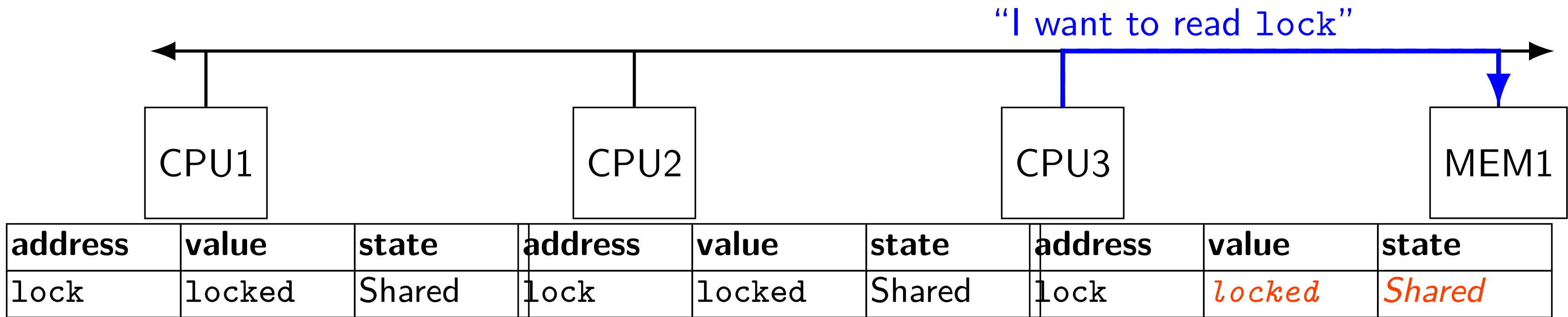
CPU2 reads lock
(to see it is still locked)

less ping-ponging



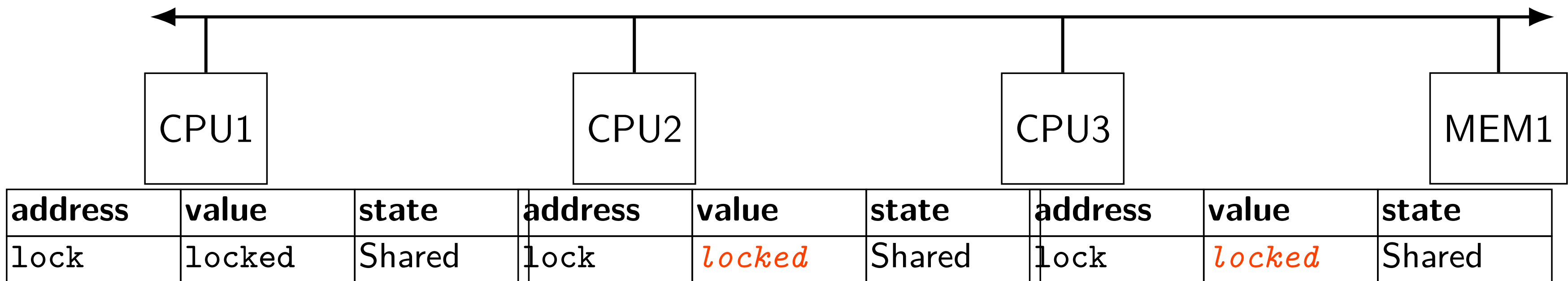
CPU1 writes back lock value,
then CPU2 reads it

less ping-ponging



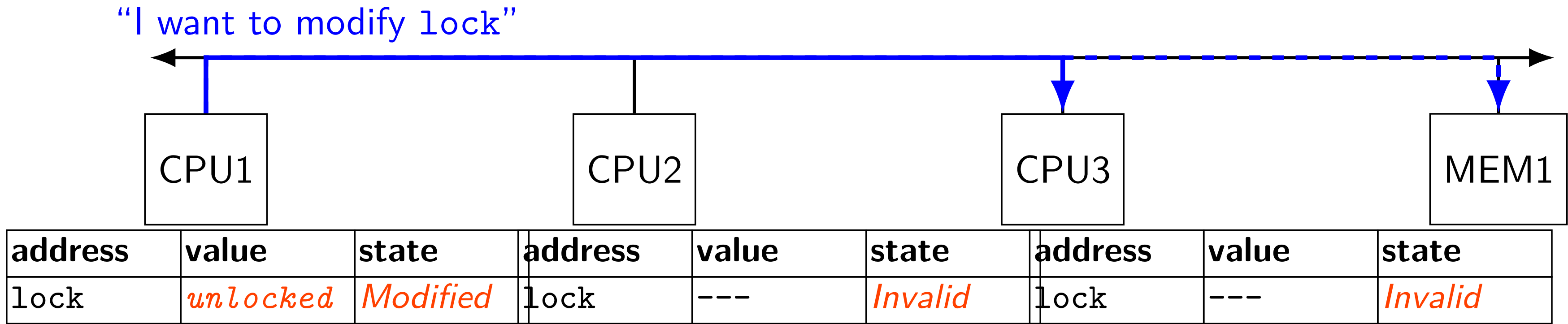
CPU3 reads lock
(to see it is still locked)

less ping-ponging



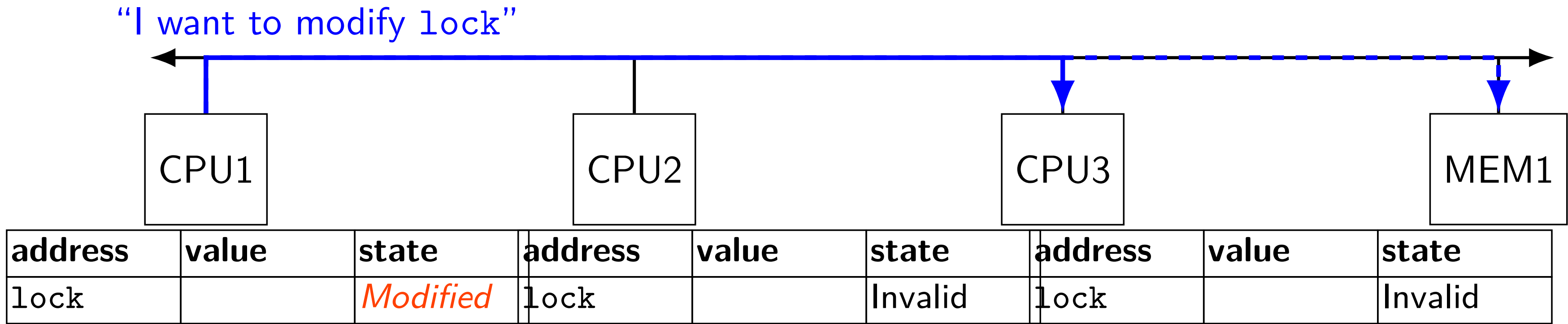
CPU2, CPU3 continue to read lock from cache
no messages on the bus

less ping-ponging



CPU1 sets lock to unlocked

less ping-ponging



some CPU (this example: CPU2) acquires lock
(CPU1 writes back value, then CPU2 reads + modifies it)

couldn't the read-modify-write instruction

notice that the value of the lock isn't changing...

and keep it in the shared state

maybe — but extra step in “common” case
(swapping different values)

more room for improvement?

can still have a lot of attempts to modify locks after unlocked

there other spinlock designs that avoid this

- ticket locks

- MCS locks

- ...

misc Linux lock stuff

Linux futexes

futex — *f*ast *u*space mut*ex*

goal: implement waiting like ‘proper’ mutexes, but...

don’t enter kernel mode most of the time

challenge: can’t acquire lock to call scheduler from user mode

futex operations

```
futex(&lock_value, FUTEX_WAIT, expected_value, ...);
```

check if `lock_value` is `expected_value`

if not — return immediately

otherwise, sleep until it `futex(..., FUTEX_WAKE` is called

```
futex(&lock_value, FUTEX_WAKE, num_processes);
```

wakeup up to `num_processes` which called `FUTEX_WAIT`

mutexes with futexes

```
int lock_value; // UNLOCKED or LOCKED_NO_WAITERS or LOCKED_WAITERS
Lock() {
retry:
    if (CompareAndSwap(&lock_value, UNLOCKED, LOCKED_NO_WAITERS) == SET) {
        /* acquired lock */
        return;
    } else if (CompareAndSwap(&lock_value, LOCKED_NO_WAITERS, LOCKED_WAITERS) == SET) {
        futex(&lock_value, FUTEX_WAIT, LOCKED_WAITERS, ...);
    }
    goto retry;
}
Unlock() {
    if (CompareAndSwap(&lock_value, LOCKED_NO_WAITERS, UNLOCKED) == SET) {
        return;
    } else {
        lock_value = UNLOCKED;
        futex(&lock_value, FUTEX_WAKE, 1, ...);
    }
}
```

implementing futex_wait

hashtable: address → queue of waiting threads

use hashtable to look-up queue

lock queue

check value hasn't changed

if so abort, releasing lock

add thread to queue

set thread as WAITING (not runnable)

unlock queue

call scheduler

woken up — queue used to set RUNNABLE

fairer spinlocks

so far — everything on spinlocks

- mutexes, condition variables — built with spinlocks

spinlocks are pretty ‘unfair’

- where fair = get lock if waiting longest

last CPU that held spinlock more likely to get it again

- already has the lock in its cache...

but there are many other ways to spinlocks...

ticket spinlocks

```
unsigned int serving_number;  
unsigned int next_number;
```

```
Lock() {  
    // "take a number"  
    unsigned int my_number = atomic_read_and_increment(&next_number);  
    // wait until "now serving" that number  
    while (atomic_read(&serving_number) != my_number) {  
        /* do nothing */  
    }  
    // MISSING: code to prevent reordering reads/writes  
}
```

```
Unlock() {  
    // serve next number  
    serving_number += 1;  
    // MISSING: code to prevent reordering reads/writes  
}
```

ticket spinlocks and cache contention

still have contention to write next_number

... but no retrying writes!

should limit 'ping-ponging'?

threads loop performing a read repeatedly while waiting

value will be broadcasted to all processors

'free' if using a bus

not-so-free if another way of connecting CPUs

beyond ticket spinlocks

Linux kernel used to use ticket spinlocks

now uses variant of MCS spinlocks — locks have linked-list queue!

- careful use of atomic operations to modify queue

still try

goal: even less contention

- unlocking value doesn't require broadcasting to all CPUs

- each processor waits on its own cache block

Backup slides

xv6 interrupt disabling (1)

```
...
acquire(struct spinlock *lk) {
    pushcli(); // disable interrupts to avoid deadlock
    ... /* this part basically just for multicore */
}
release(struct spinlock *lk)
{
    ... /* this part basically just for multicore */
    popcli();
}
```

xv6 push/popcli

pushcli / popcli — need to be in pairs

pushcli — disable interrupts if not already

popcli — enable interrupts if corresponding pushcli disabled them

don't enable them if they were already disabled

xv6 interrupt disabling: detail (3)

```
pushcli(void)
{
    int eflags;
    eflags = readeflags();
    cli();
    if (mycpu()->ncli == 0)
        mycpu()->intena = eflags & FL_IF;
    @2mycpu()2@->ncli += 1;
}

popcli(void)
{
    if(readeflags() & FL_IF)
        panic("popcli - interruptible");
    if(--@2mycpu()2@->ncli < 0)
        panic("popcli");
    if(@2mycpu()2@->ncli == 0 && @2mycpu()2@->@3intena3@)
        sti();
}
```


xv6 interrupt disabling: detail (3)

```
pushcli(void)
{
    int eflags;
    eflags = readeflags();
    cli();
    if (mycpu()->ncli == 0)
        mycpu()->intena = eflags & FL_IF;
    @2mycpu()2@->ncli += 1;
}

popcli(void)
{
    if(readeflags() & FL_IF)
        panic("popcli - interruptible");
    if(--@2mycpu()2@->ncli < 0)
        panic("popcli");
    if(@2mycpu()2@->ncli == 0 && @2mycpu()2@->@3intena3@)
        sti();
}
```

xv6 interrupt disabling: detail (3)

```
pushcli(void)
{
    int eflags;
    eflags = readeflags();
    cli();
    if (mycpu()->ncli == 0)
        mycpu()->intena = eflags & FL_IF;
    @2mycpu()2@->ncli += 1;
}
```

```
popcli(void)
{
    if(readeflags() & FL_IF)
        panic("popcli - interruptible");
    if(--@2mycpu()2@->ncli < 0)
        panic("popcli");
    if(@2mycpu()2@->ncli == 0 && @2mycpu()2@->@3intena3@)
        sti();
}
```

mycpu() — per-core information

xv6 interrupt disabling: detail (3)

```
pushcli(void)
{
    int eflags;
    eflags = readeflags();
    cli();
    if (mycpu()->ncli == 0)
        mycpu()->intena = eflags & FL_IF;
    @2mycpu()2@->ncli += 1;
}
```

```
popcli(void)
{
    if(readeflags() & FL_IF)
        panic("popcli - interruptible");
    if(--@2mycpu()2@->ncli < 0)
        panic("popcli");
    if(@2mycpu()2@->ncli == 0 && @2mycpu()2@->@3intena3@)
        sti();
}
```

intena — were interrupts enabled before first pushcli()?

xv6 interrupt disabling: detail (3)

```
pushcli(void)
{
    int eflags;
    eflags = readeflags();
    cli();
    if (mycpu()->ncli == 0)
        mycpu()->intena = eflags & FL_IF;
    @2mycpu()2@->ncli += 1;
}
```

```
popcli(void)
{
    if(readeflags() & FL_IF)
        panic("popcli - interruptible");
    if(--@2mycpu()2@->ncli < 0)
        panic("popcli");
    if(@2mycpu()2@->ncli == 0 && @2mycpu()2@->@3intena3@)
        sti();
}
```

ncli — # calls to pushcli - # calls to popcli
intended usage: each pushcli has matching popcli

xv6 interrupt disabling: detail (3)

```
pushcli(void)
{
    int eflags;
    eflags = readeflags();
    cli();
    if (mycpu()->ncli == 0)
        mycpu()->intena = eflags & FL_IF;
    @2mycpu()2@->ncli += 1;
}
```

```
popcli(void)
{
    if(readeflags() & FL_IF)
        panic("popcli - interruptible");
    if(--@2mycpu()2@->ncli < 0)
        panic("popcli");
    if(@2mycpu()2@->ncli == 0 && @2mycpu()2@->@3intena3@)
        sti();
}
```

pushcli — always disable interrupts

xv6 interrupt disabling: detail (3)

```
pushcli(void)
{
    int eflags;
    eflags = readeflags();
    cli();
    if (mycpu()->ncli == 0)
        mycpu()->intena = eflags & FL_IF;
    @2mycpu()2@->ncli += 1;
}
```

```
popcli(void)
{
    if(readeflags() & FL_IF)
        panic("popcli - interruptible");
    if(--@2mycpu()2@->ncli < 0)
        panic("popcli");
    if(@2mycpu()2@->ncli == 0 && @2mycpu()2@->@3intena3@)
        sti();
}
```

popcli — renable interrupts if last popcli
(and interrupts were enabled before)
(each pushcli had a matching popcli call)

xv6 spinlock: acquire

```
void
acquire(struct spinlock *lk)
{
    pushcli(); // disable interrupts to avoid deadlock.
    ...
    // The xchg is atomic.
    while(xchg(&lk->locked, 1) != 0)
        ;

    // Tell the C compiler and the processor to not move loads or stores
    // past this point, to ensure that the critical section's memory
    // references happen after the lock is acquired.
    __sync_synchronize();
    ...
}
```

xv6 spinlock: acquire

```
void
acquire(struct spinlock *lk)
{
    pushcli(); // disable interrupts to avoid deadlock.
    ...
    // The xchg is atomic.
    while(xchg(&lk->locked, 1) != 0)
        ;

    // Tell the C compiler and the processor to not move loads or stores
    // past this point, to ensure that the critical section's memory
    // references happen after the lock is acquired.
    __sync_synchronize();
    ...
}
```


xv6 spinlock: acquire

```
void
acquire(struct spinlock *lk)
{
    pushcli(); // disable interrupts to avoid deadlock.
    ...
    // The xchg is atomic.
    while(xchg(&lk->locked, 1) != 0)
        ;

    // Tell the C compiler and the processor to not move loads or stores
    // past this point, to ensure that the critical section's memory
    // references happen after the lock is acquired.
    __sync_synchronize();
    ...
}
```

don't let us be interrupted after while have the lock
problem: interruption might try to do something with the lock
...but that can never succeed until we release the lock
...but we won't release the lock until interruption finishes

xv6 spinlock: acquire

```
void
acquire(struct spinlock *lk)
{
    pushcli(); // disable interrupts to avoid deadlock.
    ...
    // The xchg is atomic.
    while(xchg(&lk->locked, 1) != 0)
        ;

    // Tell the C compiler and the processor to not move loads or stores
    // past this point, to ensure that the critical section's memory
    // references happen after the lock is acquired.
    __sync_synchronize();
    ...
}
```

xchg wraps the lock xchg instruction
same loop as before

xv6 spinlock: acquire

```
void
acquire(struct spinlock *lk)
{
    pushcli(); // disable interrupts to avoid deadlock.
    ...
    // The xchg is atomic.
    while(xchg(&lk->locked, 1) != 0)
        ;

    // Tell the C compiler and the processor to not move loads or stores
    // past this point, to ensure that the critical section's memory
    // references happen after the lock is acquired.
    __sync_synchronize();
    ...
}
```

avoid load store reordering (including by compiler)
on x86, xchg alone is enough to avoid processor's reordering
(but compiler may need more hints)

xv6 spinlock: release

```
void
release(struct spinlock *lk)
{
    ...
    // Tell the C compiler and the processor to not move loads or stores
    // past this point, to ensure that all the stores in the critical
    // section are visible to other cores before the lock is released.
    // Both the C compiler and the hardware may re-order loads and
    // stores; __sync_synchronize() tells them both not to.
    __sync_synchronize();

    // Release the lock, equivalent to lk->locked = 0.
    // This code can't use a C assignment, since it might
    // not be atomic. A real OS would use C atomics here.
    asm volatile("movl $0, %0" : "+m" (lk->locked) : );

    popcli();
}
```

xv6 spinlock: release

```
void
release(struct spinlock *lk)
{
    ...
    // Tell the C compiler and the processor to not move loads or stores
    // past this point, to ensure that all the stores in the critical
    // section are visible to other cores before the lock is released.
    // Both the C compiler and the hardware may re-order loads and
    // stores; __sync_synchronize() tells them both not to.
    __sync_synchronize();

    // Release the lock, equivalent to lk->locked = 0.
    // This code can't use a C assignment, since it might
    // not be atomic. A real OS would use C atomics here.
    asm volatile("movl $0, %0" : "+m" (lk->locked) : );

    popcli();
}
```


xv6 spinlock: release

```
void
release(struct spinlock *lk)
{
    ...
    // Tell the C compiler and the processor to not move loads or stores
    // past this point, to ensure that all the stores in the critical
    // section are visible to other cores before the lock is released.
    // Both the C compiler and the hardware may re-order loads and
    // stores; __sync_synchronize() tells them both not to.
    __sync_synchronize();

    // Release the lock, equivalent to lk->locked = 0.
    // This code can't use a C assignment, since it might
    // not be atomic. A real OS would use C atomics here.
    asm volatile("movl $0, %0" : "+m" (lk->locked) : );

    popcli();
}
```

turns into instruction to tell processor not to reorder
plus tells compiler not to reorder

xv6 spinlock: release

```
void
release(struct spinlock *lk)
{
    ...
    // Tell the C compiler and the processor to not move loads or stores
    // past this point, to ensure that all the stores in the critical
    // section are visible to other cores before the lock is released.
    // Both the C compiler and the hardware may re-order loads and
    // stores; __sync_synchronize() tells them both not to.
    __sync_synchronize();

    // Release the lock, equivalent to lk->locked = 0.
    // This code can't use a C assignment, since it might
    // not be atomic. A real OS would use C atomics here.
    asm volatile("movl $0, %0" : "+m" (lk->locked) : );

    popcli();
}
```

turns into mov of constant 0 into lk->locked

xv6 spinlock: release

```
void
release(struct spinlock *lk)
{
    ...
    // Tell the C compiler and the processor to not move loads or stores
    // past this point, to ensure that all the stores in the critical
    // section are visible to other cores before the lock is released.
    // Both the C compiler and the hardware may re-order loads and
    // stores; __sync_synchronize() tells them both not to.
    __sync_synchronize();

    // Release the lock, equivalent to lk->locked = 0.
    // This code can't use a C assignment, since it might
    // not be atomic. A real OS would use C atomics here.
    asm volatile("movl $0, %0" : "+m" (lk->locked) : );

    popcli();
}
```

reenable interrupts (taking nested locks into account)

xv6 spinlock: debugging stuff

```
void acquire(struct spinlock *lk) {
    ...
    if(holding(lk))
        panic("acquire")
    ...
    // Record info about lock acquisition for debugging.
    lk->cpu = mycpu();
    getcallerpcs(&lk, lk->pcs);
}

void release(struct spinlock *lk) {
    if(!holding(lk))
        panic("release");

    lk->pcs[0] = 0;
    lk->cpu = 0;
    ...
}
```

xv6 spinlock: debugging stuff

```
void acquire(struct spinlock *lk) {
    ...
    if(holding(lk))
        panic("acquire")
    ...
    // Record info about lock acquisition for debugging.
    lk->cpu = mycpu();
    getcallerpcs(&lk, lk->pcs);
}
void release(struct spinlock *lk) {
    if(!holding(lk))
        panic("release");

    lk->pcs[0] = 0;
    lk->cpu = 0;
    ...
}
```

xv6 spinlock: debugging stuff

```
void acquire(struct spinlock *lk) {
    ...
    if(holding(lk))
        panic("acquire")
    ...
    // Record info about lock acquisition for debugging.
    lk->cpu = mycpu();
    getcallerpcs(&lk, lk->pcs);
}

void release(struct spinlock *lk) {
    if(!holding(lk))
        panic("release");

    lk->pcs[0] = 0;
    lk->cpu = 0;
    ...
}
```

xv6 spinlock: debugging stuff

```
void acquire(struct spinlock *lk) {
    ...
    if(holding(lk))
        panic("acquire")
    ...
    // Record info about lock acquisition for debugging.
    lk->cpu = mycpu();
    getcallerpcs(&lk, lk->pcs);
}
void release(struct spinlock *lk) {
    if(!holding(lk))
        panic("release");

    lk->pcs[0] = 0;
    lk->cpu = 0;
    ...
}
```